

PORYGON: A Statistical and Rule-Based Runtime Security Framework for Containerized Workloads

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: 75

S.No	Register Number	Student Name
1	CB.EN.U4CSE23504	Ashin Varghese
2	CB.EN.U4CSE23559	Anurup Krishna
3	CB.EN.U4CSE23628	Krishnan S
4	CB.EN.U4CSE23661	Madhav Sreejith

Guide: **Dr Abirami K**, Assistant Professor, Department of CSE

September 6, 2026

Research Question and Contribution

The specific, falsifiable question this project answers

The unanswered design question: container security tools learn what “normal” process behavior looks like, then flag deviations. But how finely should “normal” be defined? Four real options, in increasing order of specificity:

- ❶ **One-size-fits-all** – a single baseline shared by every container regardless of what image it runs.
- ❷ **Per mutable label** – one baseline per image tag (e.g. `myapp:latest`); the risk is that a tag can be silently repointed to different image content later.
- ❸ **Per exact image** – one baseline per cryptographic image identity, which cannot silently change.
- ❹ **Per exact image + configuration** – the same, but also split by the container’s specific security settings (permissions, capabilities).

No existing tool or published study (Falco, Sysdig, Aqua, or the academic line from Forrest et al. 1996 through Abed/Clancy 2015) measures which of these four actually reduces false alarms. This project does, using 216 real container trials and formal statistical testing rather than a single accuracy figure.

Result: option 3 eliminated false alarms entirely versus option 1 (100% $\rightarrow$ 0%), without missing any real attack (96/96 caught either way). This was not a fluke – the same result appeared at two different sample sizes ($p = 5.8 \times 10^{-11}$ at 144 trials, $p = 6.7 \times 10^{-16}$ at 216 trials), and got statistically stronger, not weaker, the second time.

1. Methodology & Solution Design

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Methodological Framework:

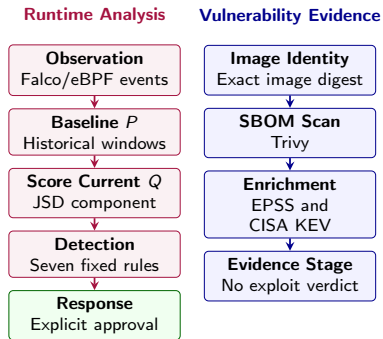
- **Behaviour Baseline:** Historical Docker lifecycle and Falco process events form a versioned baseline for one immutable image digest.
- **Anomaly Scoring:** For each categorical feature, the v1 scorer compares baseline P with a completed runtime window Q using base-2 Jensen–Shannon distance:

$$D_{JS}(P, Q) = \sqrt{JSD(P, Q)}$$

Smaller values indicate more similar behaviour.

- **Rule-Based Detection:** Seven deterministic rules produce explainable findings. An incident requires an unsuppressed, incident-eligible finding.

Porygon Workflow



1. Methodology & Solution Design (contd.)

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Implementation Approach and Design Choices:

- **Kernel-Level Monitoring:** Falco's modern eBPF driver captures container process execution without modifying the applications. The Telemetry Adapter ingests those events, while the Docker Collector records lifecycle and image-identity events. Both use bounded SQLite outboxes with retry delivery.
- **Service Integration:** Docker Compose coordinates eight long-running containers. Segregated networks restrict connectivity, and outbound access is limited to the vulnerability scanner.
- **Explainable Analysis:** Base-2 Jensen–Shannon distance provides a bounded, symmetric measure of categorical change. Fixed rules expose the matched evidence, so the implemented path does not depend on a black-box ML classifier.
- **Controlled Response:** Policy thresholds use severity and confidence to determine allowed containment actions. Disruptive execution is disabled by default. When enabled, it still requires explicit human approval.
- **Implementation Verification:** The repository contains 79 backend test functions across baseline, scoring, detection, calibration, response, and vulnerability modules, plus 131 further test functions across the collector, telemetry, responder, and scanner services (210 total).

2. Implementation Progress & Module Functionality

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

- **Implementation Status Overview:**

Module / Component	Current Implementation State	Status
Kernel Telemetry & Ingestion (Falco eBPF, Docker Collector, Telemetry Adapter)	Captures container process execution via eBPF; durable SQLite outbox with retry	Functional
Behavioural Baseline & JSD Scoring (baseline.py, scoring.py)	Digest-bound behavioural baselines; JSD-based anomaly distance scoring	Functional
Deterministic Detection Engine (detection.py)	7 correlation rules (POR-DET-001-007) generating findings/incidents	Functional
4-Arm Scope Comparison (experiments/)	216-trial confirmatory run analysed end to end, exact statistical pipeline	Complete

2. Implementation Progress (contd.)

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module / Component	Current Implementation State	Status
Vulnerability & SBOM Pipeline (Scanner, Trivy 0.72.0)	CycloneDX SBOM generation; EPSS & CISA KEV enrichment	Functional
Response Policy Engine (response.py)	Severity/confidence-gated containment recommendations; human-approval required	Functional
Module Integration	8 containerized services integrated via Docker Compose with network segregation	Integrated

Core Functionality Demonstration: A real, functional XMRig cryptominer binary from a published malware-sample container image was executed against a live Porygon baseline built from 243 real Falco-captured events, with zero prior knowledge of the binary given to the system. Resulting anomaly score: 0.2696 (elevated band, threshold 0.25), with the executable and its parent-child call edge both flagged as previously unseen. Full record: docs/LIVE_DEMO_RECORD_V1.md.

3. Technical Knowledge & Design Decisions

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Distance metric selection: Jensen-Shannon divergence (JSD) over Kullback-Leibler (KL) divergence. KL divergence is asymmetric ($D_{KL}(P, Q) \neq D_{KL}(Q, P)$) and unbounded above, so a KL score computed for one container cannot be compared against a KL score from a different container on a common scale. JSD is the symmetric, bounded variant ($JSD(P, Q) = JSD(Q, P)$, range $[0, 1]$), which permits one fixed threshold to apply uniformly across all containers.

Containment decision logic. A finding does not trigger any action unless the rule engine's severity (*Sev*) and confidence (*Conf*) scores clear a defined threshold. Pause requires $Sev \geq 0.70$ and $Conf \geq 0.55$. Stop requires $Sev \geq 0.90$, $Conf \geq 0.75$, and a match on one of two specific high-confidence rules (unseen-shell-to-tool sequence, or a container transitioning to privileged mode). Every action, at either threshold, requires explicit human operator approval before execution; no threshold triggers automatic execution.

Statistical significance of the primary result. On 216 real container trials, per-image baseline scoping reduced the false-alarm rate from 100% to 0% relative to a single pooled baseline, with zero reduction in attack detection. The exact statistical test applied to this result returns $p = 6.7 \times 10^{-16}$, several orders of magnitude below the conventional $p < 0.05$ significance threshold.

3. Technical Knowledge & Design Decisions (contd.)

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Multicall binary handling. Utilities such as BusyBox and Toybox implement dozens of commands inside a single executable file; the command executed depends on the invocation name, not the file path. Tracking file identity alone would classify every invoked command as the same previously-seen program. The detection engine instead tracks the invocation (command) name, so a new command invoked through an already-known multicall binary is still flagged as novel.

Canonical scan identity. Each vulnerability scan result is keyed by a SHA-256 hash of six inputs: image digest, local image ID, scanner name, scanner version, vulnerability database schema version, and a researcher-supplied scan reference. Re-executing a scan with an identical set of these six inputs returns the existing stored result (idempotent). A change to any one input produces a distinct, separately-tracked scan record (immutable).

Network segregation. The eight services are partitioned across three isolated Docker networks: ingress (external traffic), internal (inter-service communication), and egress (outbound-only, used exclusively by the vulnerability scanner). A service is not granted network reachability to segments outside its assigned role.

Versioned scoring configuration. The scoring algorithm and its numeric weights are bound to a named algorithm version. A weight modification requires incrementing the version identifier; no weight change is applied in place. Every historical score therefore remains reproducible against the exact configuration that produced it.

3. Technical Knowledge & Design Decisions (contd.)

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

- **Confirmed Findings (216 real trials, exact statistical pipeline):**

- Per-image scoping vs. pooled baseline: significant, $p = 6.7 \times 10^{-16}$, FPR 100% $\rightarrow$ 0%, recall unchanged (96/96 both arms).
- Digest-plus-context vs. digest-only: no measurable difference in this dataset, $p = 1.0$, reproduced at both $n = 35$ and $n = 53$ – traced to the tested configuration change (a dropped Linux capability) not altering which processes execute, which the process-name feature cannot see by construction.
- Follow-on feature scoring the security-configuration document directly separates every tested case cleanly: 0/26 false positives, 27/27 correct detections.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- **Individual MOOC / Online Certification Selection:**

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Ashin Varghese	[Course Title relevant to Project]	Coursera	15 Hrs
2	Anurup Krishna	[Course Title relevant to Project]	NPTEL	20 Hrs
3	Krishnan S	[Course Title relevant to Project]	edX / Coursera	12 Hrs
4	Madhav Sreejith	[Course Title relevant to Project]	AWS / Google	16 Hrs

- **Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.

References I



S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symp. Security and Privacy*, 1996, pp. 120–128.



S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.



A. S. Abed, T. C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–5.



J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.



A. Salem and F. Naït-Abdesselam, "Anomaly detection in network traffic using Jensen-Shannon divergence," in *Proc. IEEE International Conference on Communications (ICC)*, 2012.



X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, "eBPF-Guard: a detection method for container escape via multi-level monitoring and enhanced analysis model," *Empirical Software Engineering*, vol. 31, article 51, 2026.



Li Wei, Yuan Zekun, Wu Kehe, and Cheng Rui, "Container anomaly detection based on attention mechanism and multiscale convolutional neural network," *Journal of Information Security Research*, vol. 11, no. 1, pp. 35–, 2025.



K. Xiong, Z. Wu, and X. Jia, "DeepContainer: a deep learning-based framework for real-time anomaly detection in cloud-native container environments," *Journal of Advanced Computing Systems*, 2025.

References II



J. H. Ring, C. M. Van Oort, S. Durst, V. White, J. P. Near, and C. Skalka, "Methods for host-based intrusion detection with deep learning," *Digital Threats: Research and Practice*, vol. 2, no. 4, 2021.



B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You